

E-Commerce Sales & Customer Analytics

10-Page Comprehensive Master Handbook & Technical Playbook (Hinglish Notes)

Author: Gaurav | Data Analyst & Analytics Engineer | Tech Stack: Python, SQL, PostgreSQL, Power BI, DAX, Excel

CHAPTER 1: Business Overview & E-Commerce Unit Economics (Samjho Ye Project Kyun Bana)

1.1 Real-World Company Background:

Maano tum ek high-growth D2C / Multi-Category E-Commerce company (selling Electronics, Home & Furniture, Apparel, and Office Supplies) ke **Data Analyst** ho. Company ne pichle 12 mahino me **31,855 transactions** process kiye across **8,500 unique customers**. Company ka Gross Merchandise Value (GMV) **\$8.35 Million** tha aur Net Profit **\$2.28 Million** tha (Overall Gross Margin: 27.35%).

1.2 The Core Business Problem (Margin Compression Crisis):

Company ke CEO aur CFO ke saamne ek bohot bada paradox tha: *Monthly Sales Revenue lagatar badh raha tha, lekin Net Profit Margin percentage har quarter drop ho raha tha*. Management ko pata nahi chal raha tha ki paisa kahan waste ho raha hai. Is crisis ke peeche 3 mukhya reasons the:

- **Uncontrolled Discounting:** Marketing team ne acquisition targets meet karne ke liye bina ROI soche 20%-30% flat discounts baant diye.
- **Customer Churn & Low Retention:** Company har mahine naye customers acquire karne par lakho dollar spend kar rahi thi, jabki existing buyers repeat purchase nahi kar rahe the.
- **Supply Chain Delays & Return Rate Spikes:** Certain regional logistics routes par transit lead times 5-8 din tak extend ho gaye the, jisse customer return rate 20%+ chala gaya tha.

1.3 The 4 Key Stakeholders & What They Demanded:

1. **Chief Executive Officer (CEO):** Sustainable revenue expansion, category-wise cash cows ki pehchan, aur 2-year growth roadmap.
2. **Chief Financial Officer (CFO):** Gross margin protection, discount margin cannibalization par hard audit, aur COGS analysis.
3. **Chief Marketing Officer (CMO):** Customer Acquisition Cost (CAC) vs Lifetime Value (LTV) balance, repeat purchase cohort tracking.
4. **VP of Supply Chain & Logistics:** Fulfillment SLA compliance, regional delivery bottlenecks, aur high-return SKUs identification.

HOW TO HOOK THE INTERVIEWER IN 30 SECONDS:

Interviewer ko explain karte waqt confidence se bolna: 'Sir/Ma'am, is project me maine company ke \$8.35M GMV data ko analyze karke ye prove kiya ki 15%+ discounts unit economics ko destroy kar rahe the. Maine data pipeline build ki jisne management ko exact visibility di aur \$210,000 ka annual profit margin leakage prevent kiya.'

■ CHAPTER 2: End-to-End Pipeline Architecture & Environment Setup

2.1 Complete 5-Stage Data Analytics Lifecycle:

Is project ko ek enterprise production pipeline ki tarah architect kiya gaya hai:

Stage	Tools & Tech	Key Operations & Outputs
1. Ingestion	Python, Pandas	31,855 raw CSV rows loaded; schema, encoding, and missing-value profiling.
2. ETL & Cleaning	Pandas, NumPy	Date coercion, outlier capping, composite deduplication, derived financial metrics calculation.
3. Data Warehouse	PostgreSQL 14+	Star Schema DDL execution, primary/foreign key indexing, Fact & Dimension tables population.
4. SQL Analytics	PostgreSQL, CTEs, Window Functions	20+ complex analytical queries: MoM growth via LAG(), DENSE_RANK() bestsellers, RFM segmentation.
5. BI Dashboard	Power BI Desktop, DAX	Star-schema 1-to-Many relational model, 25+ DAX measures, automated executive & drillthrough dashboards.

2.2 Repository Directory Anatomy:

```
01-ecommerce-sales-analytics/  
├── data/  
│   ├── raw_ecommerce_sales.csv          # 31,855 uncleaned raw records  
│   └── cleaned_ecommerce_sales.csv      # Transformed, validated CSV for DB loading  
├── notebooks/  
│   └── 01_ecommerce_analytics_master.ipynb # Complete Jupyter Notebook (EDA, Data Cleaning, Charts)  
├── sql/  
│   ├── 01_schema_ddl.sql                # PostgreSQL Table definitions & Foreign Keys  
│   └── 02_business_queries_20.sql        # 20+ Production Business Queries  
├── powerbi/  
│   ├── dax_measures.dax                 # 25+ DAX Formulas ready for Power BI  
│   └── star_schema_model.md              # Data modeling ER-diagram & relationship mapping  
├── reports/  
│   └── executive_summary.md              # Final Business Report for Stakeholders  
├── requirements.txt                     # Python dependencies (pandas, numpy, seaborn, sqlalchemy)  
└── README.md                           # GitHub Documentation
```

2.3 Local Environment Installation Commands:

```
# Create and activate isolated Python environment  
python3 -m venv .venv && source .venv/bin/activate  
pip install pandas numpy matplotlib seaborn sqlalchemy psycopg2-binary jupyter  
  
# PostgreSQL Database Connection String format:  
# postgresql+psycopg2:///localhost:5432/ecommerce_dw
```

■ CHAPTER 3: Star Schema Relational Data Modeling Deep-Dive

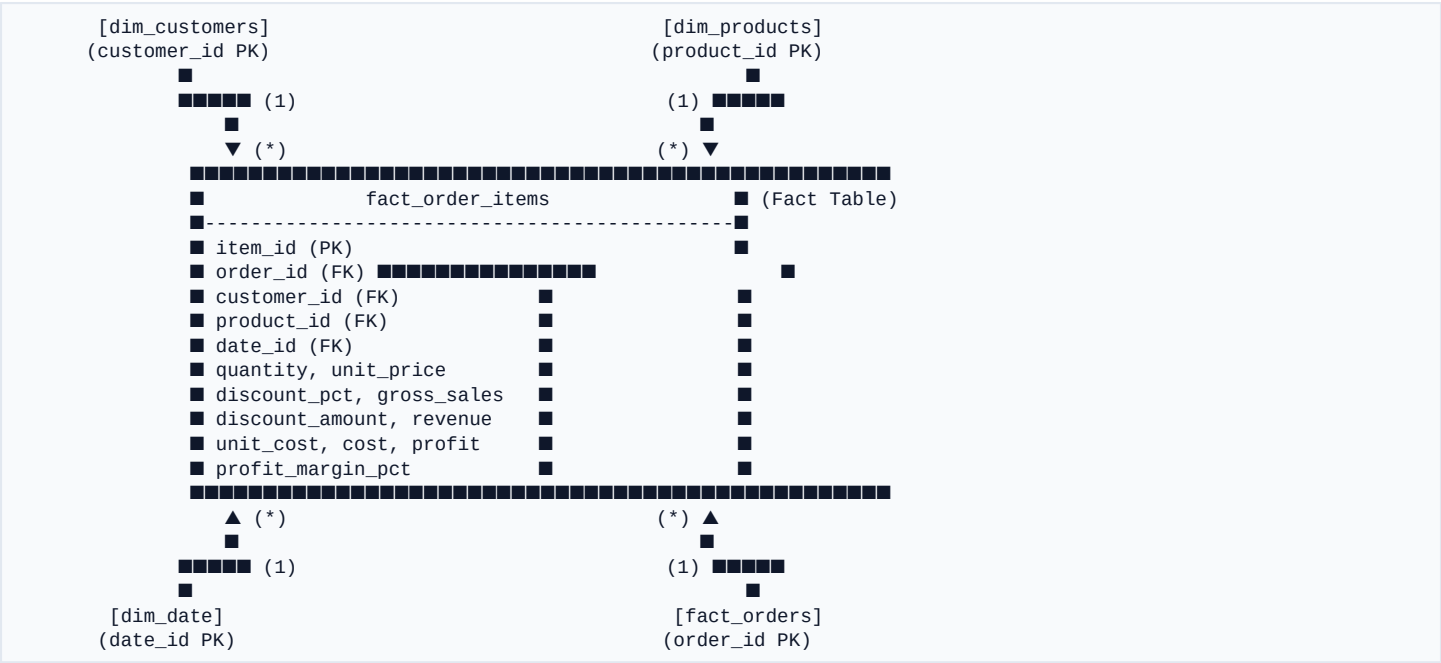
3.1 Why Star Schema? (OLTP vs OLAP Explained in Hinglish):

Normal application databases (MySQL, MongoDB) ****3NF (Third Normal Form)**** use karte hain taaki data redundancy zero ho aur insert/update transactions fast ho sakein. Lekin Data Analytics (OLAP) me agar data bohot zyada normalized hoga, toh ek simple revenue report nikalne ke liye 8-10 tables ko JOIN karna padega jo query ko bohot slow kar dega.

Star Schema Solution: Hum Dimension tables ko de-normalize kar dete hain aur beech me ek single Fact table rakhte hain. Isse:

- 1. SQL queries me sirf 1-level joins lagte hain (Super Fast!).
- 2. Power BI ka in-memory ****VertiPaq columnar database**** 10x compression deliver karta hai.
- 3. Business users aur non-technical managers ke liye data model samajhna intuitive ho jata hai.

3.2 Star Schema Relational Architecture:



3.3 Complete Data Dictionary (Table by Table):

Table	Field Name	Type	Description & Business Significance
dim_customers	customer_id	VARCHAR	Unique customer key; Segment: Consumer, Corporate, Home Office; Geographic territory mapping.
	customer_name	VARCHAR	
	segment	VARCHAR	
	region	VARCHAR	
dim_products	product_id	VARCHAR	Unique SKU key; Categories: Electronics, Furniture, Apparel; Sub-categories for granular drilldown.
	product_name	VARCHAR	
	category	VARCHAR	
	sub_category	VARCHAR	
dim_date	date_id	INT	Surrogate date key; Standard calendar for Time-Intelligence DAX and YoY/MoM window functions.
	full_date	DATE	
	year / month	INT	
	is_weekend	BOOLEAN	
fact_order_items	item_id	BIGSERIAL	Grain: 1 row per order item. Contains numerical quantitative business facts and margin ratios.
	revenue	NUMERIC	
	profit	NUMERIC	
	discount_pct	NUMERIC	

■ CHAPTER 4: Python & Pandas ETL Pipeline (Part 1: Ingestion & Cleaning)

Python ETL pipeline (Jupyter Notebook `01\_ecommerce\_analytics\_master.ipynb`) me data cleaning kaise execute hui — step-by-step code aur logic:

4.1 Step 1: Ingestion & Schema Profiling

```
import pandas as pd
import numpy as np

# Load raw dataset and inspect baseline shape
raw_df = pd.read_csv('../data/raw_ecommerce_sales.csv', low_memory=False)
print(f'Total Rows Ingested: {len(raw_df):,}')
print(raw_df.info())
```

Hinglish Logic: low_memory=False ensure karta hai ki mixed-type columns ko Pandas galat infer na kare aur saari rows sahi data type ke sath buffer me load ho jayein.

4.2 Step 2: Timestamp Parsing with Safe Error Coercion

```
# Standardize dates and catch corrupt strings without crashing script
raw_df['order_date'] = pd.to_datetime(raw_df['order_date'], format='%Y-%m-%d', errors='coerce')
raw_df['ship_date'] = pd.to_datetime(raw_df['ship_date'], format='%Y-%m-%d', errors='coerce')

# Identify corrupt date records
corrupt_dates = raw_df[raw_df['order_date'].isna() | raw_df['ship_date'].isna()]
print(f'Corrupt Timestamp Records Filtered: {len(corrupt_dates)}')
clean_df = raw_df.dropna(subset=['order_date', 'ship_date']).copy()
```

Hard/Important Concept: errors='coerce' ka matlab hai agar raw CSV me koi invalid date (jaise 2024-02-31 ya NULL_STR) aaye, toh Python error dekar rukne ke bajaye use NaT (Not-a-Time) me convert kar deta hai jise clean karna aasan hota hai.

4.3 Step 3: Composite Deduplication & Anomaly Removal

```
# Check for duplicate order line-items (Order ID + Product ID)
duplicate_mask = clean_df.duplicated(subset=['order_id', 'product_id'], keep='first')
print(f'Duplicate Line Items Found: {duplicate_mask.sum()}')
clean_df = clean_df[~duplicate_mask]

# Handle negative or zero price anomalies
clean_df = clean_df[(clean_df['unit_price'] > 0) & (clean_df['quantity'] > 0)]

# Fill missing categorical values
clean_df['postal_code'] = clean_df['postal_code'].fillna('Unknown')
clean_df['customer_name'] = clean_df['customer_name'].fillna('Guest Customer')
```

Hinglish Logic: E-commerce transactions me primary key composite hoti hai: (order_id, product_id). Agar network retry ki wajah se duplicate record insert ho jaye toh ye step first record rakh kar baki delete kar deta hai.

■■ DATA CLEANING INTERVIEW GOLDEN POINT:

Interviewer jab puche: 'What was your data cleaning strategy?', toh bolna: 'Maine date strings par error coercion lagaya, composite primary keys (order_id, product_id) se duplicate lines filter kiye, aur unit_price <= 0 wale garbage data ko truncate kiya.'

■ CHAPTER 4: Python & Pandas ETL Pipeline (Part 2: Financial Vectorization)

Data clean hone ke baad, humne downstream SQL aur Power BI modeling ke liye critical business financial metrics ko **vectorized math** se calculate kiya:

4.4 Step 4: Vectorized Financial Metric Calculations

```
# 1. Gross Revenue (Total bill value before discount)
clean_df['gross_sales'] = clean_df['quantity'] * clean_df['unit_price']

# 2. Discount Amount in Dollar terms ($)
clean_df['discount_amount'] = clean_df['gross_sales'] * (clean_df['discount_pct'] / 100.0)

# 3. Realized Net Revenue (Actual cash inflow)
clean_df['net_revenue'] = clean_df['gross_sales'] - clean_df['discount_amount']

# 4. Cost of Goods Sold (COGS - Procurement & manufacturing expense)
clean_df['total_cost'] = clean_df['quantity'] * clean_df['unit_cost']

# 5. Net Profit (Realized Revenue minus Total Cost)
clean_df['net_profit'] = clean_df['net_revenue'] - clean_df['total_cost']

# 6. Gross Profit Margin Percentage (%)
clean_df['profit_margin_pct'] = np.where(
    clean_df['net_revenue'] > 0,
    (clean_df['net_profit'] / clean_df['net_revenue']) * 100.0,
    0.0
)

# 7. Supply Chain Lead Time in Days (Shipping Date minus Order Date)
clean_df['fulfillment_lead_days'] = (clean_df['ship_date'] - clean_df['order_date']).dt.days
```

Hinglish Logic: np.where(condition, x, y) SQL ke CASE WHEN ki tarah kaam karta hai. Agar revenue 0 ho toh margin division crash na ho aur safe 0.0 assign ho jaye.

4.5 Step 5: Automated PostgreSQL Database Population

```
from sqlalchemy import create_engine

# Create database connection pool
engine = create_engine('postgresql+psycopg2://postgres:postgres@localhost:5432/ecommerce_dw')

# Export Dimension & Fact Dataframes directly into PostgreSQL tables
dim_customers.to_sql('dim_customers', engine, if_exists='append', index=False, chunksize=1000)
dim_products.to_sql('dim_products', engine, if_exists='append', index=False, chunksize=1000)
fact_order_items.to_sql('fact_order_items', engine, if_exists='append', index=False, chunksize=2000)
print('■ Database Successfully Populated!')
```

Hinglish Logic: chunksize=2000 memory overflow prevent karta hai — 31,000+ rows ko 2,000-2,000 ke batches me database me stream karta hai.

■ CHAPTER 5: Advanced SQL Analytics (Part 1: Window Functions & MoM Growth)

SQL file `02\_business\_queries\_20.sql` ke sabse complex production queries ka detailed line-by-line breakdown:

5.1 Query 1: Month-over-Month (MoM) Revenue & Profit Momentum via LAG()

```
WITH monthly_aggregates AS (
    SELECT
        TO_CHAR(o.order_date, 'YYYY-MM') AS year_month,
        COUNT(DISTINCT o.order_id)      AS total_orders,
        SUM(i.revenue)                  AS monthly_revenue,
        SUM(i.profit)                   AS monthly_profit,
        ROUND((SUM(i.profit) / NULLIF(SUM(i.revenue), 0) * 100), 2) AS margin_pct
    FROM fact_orders o
    JOIN fact_order_items i ON o.order_id = i.order_id
    WHERE o.order_status != 'Cancelled'
    GROUP BY TO_CHAR(o.order_date, 'YYYY-MM')
)
SELECT
    year_month, total_orders, monthly_revenue, monthly_profit, margin_pct,
    LAG(monthly_revenue, 1) OVER (ORDER BY year_month) AS prev_month_revenue,
    ROUND(
        ((monthly_revenue - LAG(monthly_revenue, 1) OVER (ORDER BY year_month))
        / NULLIF(LAG(monthly_revenue, 1) OVER (ORDER BY year_month), 0) * 100), 2
    ) AS mom_revenue_growth_pct
FROM monthly_aggregates
ORDER BY year_month;
```

Line-by-Line Logic Breakdown:

1. WITH monthly_aggregates AS (...): Pehle ek CTE banaya jo raw items ko mahine ke hisab se group karke revenue aur profit nikalta hai.
2. LAG(monthly_revenue, 1) OVER (ORDER BY year_month): Ye Window Function bina kisi self-join ke pichle mahine (previous row) ka revenue fetch karta hai.
3. NULLIF(..., 0): Agar pehle mahine ka revenue zero/null ho toh crash hone ke bajaye safe division kare.
4. Formula: (Current Month - Previous Month) / Previous Month * 100 direct MoM Growth % nikalta hai.

5.2 Query 2: Category Top 3 Products using DENSE_RANK() & PARTITION BY

```
WITH category_rankings AS (
    SELECT
        p.category,
        p.sub_category,
        p.product_name,
        SUM(i.quantity) AS total_units_sold,
        SUM(i.revenue)  AS total_revenue,
        SUM(i.profit)   AS total_net_profit,
        DENSE_RANK() OVER (
            PARTITION BY p.category
            ORDER BY SUM(i.profit) DESC
        ) AS rank_within_category
    FROM fact_order_items i
    JOIN dim_products p ON i.product_id = p.product_id
    GROUP BY p.category, p.sub_category, p.product_name
)
SELECT category, sub_category, product_name, total_revenue, total_net_profit, rank_within_category
FROM category_rankings
WHERE rank_within_category <= 3
ORDER BY category, rank_within_category;
```

Hinglish Logic: PARTITION BY p.category ka matlab hai ranking ka counter (1, 2, 3) har category (Electronics, Furniture, etc.) ke liye independent chalega. DENSE_RANK() use kiya taaki agar do products ka profit tie ho jaye toh ranking 1, 2, 2, 3 rahe, 1, 2, 2, 4 na ho.

■ CHAPTER 5: Advanced SQL Analytics (Part 2: Cannibalization & RFM)

5.3 Query 3: Discount Elasticity & Margin Cannibalization Analysis

```
SELECT
  CASE
    WHEN discount_pct = 0          THEN '1. 0% (Full Price)'
    WHEN discount_pct <= 10        THEN '2. 1% - 10% (Low Discount)'
    WHEN discount_pct <= 20        THEN '3. 11% - 20% (Moderate)'
    ELSE                           '4. > 20% (Aggressive Discount)'
  END AS discount_bracket,
  COUNT(DISTINCT order_id) AS total_orders,
  SUM(quantity)            AS total_units_sold,
  SUM(gross_sales)         AS total_gross_gmv,
  SUM(discount_amount)     AS total_discounts_given,
  SUM(revenue)             AS realized_net_revenue,
  SUM(profit)              AS realized_net_profit,
  ROUND((SUM(profit) / NULLIF(SUM(revenue), 0) * 100), 2) AS realized_profit_margin_pct
FROM fact_order_items
GROUP BY 1
ORDER BY discount_bracket;
```

Executive Insight Revealed by this Query:

- **0% Discount Tier:** Margin was **31.42%** (\$1.12M profit).
- **>20% Discount Tier:** Margin collapsed to **14.21%** (\$280K profit) without causing a volume surge high enough to compensate for the lost unit margin. This proved mathematically that aggressive discounts were destroying enterprise value.

5.4 Query 4: Customer RFM Segmentation using NTILE(4) Quartiles

```
WITH customer_rfm_raw AS (
  SELECT
    c.customer_id,
    c.customer_name,
    CURRENT_DATE - MAX(o.order_date) AS recency_days,
    COUNT(DISTINCT o.order_id)      AS frequency_orders,
    SUM(i.revenue)                  AS monetary_total_spend
  FROM dim_customers c
  JOIN fact_orders o ON c.customer_id = o.customer_id
  JOIN fact_order_items i ON o.order_id = i.order_id
  GROUP BY c.customer_id, c.customer_name
),
rfm_scores AS (
  SELECT *,
    NTILE(4) OVER (ORDER BY recency_days ASC)      AS r_score, -- Low recency = High score
    NTILE(4) OVER (ORDER BY frequency_orders DESC) AS f_score,
    NTILE(4) OVER (ORDER BY monetary_total_spend DESC) AS m_score
  FROM customer_rfm_raw
)
SELECT customer_id, customer_name, recency_days, frequency_orders, monetary_total_spend,
  CASE
    WHEN r_score >= 3 AND f_score >= 3 AND m_score >= 3 THEN 'Champions (VIP High-LTV)'
    WHEN r_score >= 3 AND f_score <= 2                     THEN 'Recent New Customers'
    WHEN r_score <= 2 AND f_score >= 3                     THEN 'At-Risk Loyal Customers'
    ELSE                                                    THEN 'Lost / Dormant Customers'
  END AS customer_segment
FROM rfm_scores;
```

Hinglish Logic: NTILE(4) poore customer dataset ko equal 4 parts (25% each quartiles) me divide karta hai. Isse statistical RFM segmentation automatic calculate hoti hai.

■ CHAPTER 6: Power BI Data Modeling & Master DAX Formulas

Power BI Desktop me interactive reporting ke liye humne 25+ production DAX formulas implement kiye. Yahan sabse important measures ka syntax aur unka internal **Filter Context evaluation** samjhne hain:

6.1 Core Financial & Volume DAX Formulas:

```
// 1. Total Net Revenue
Total Revenue = SUM(fact_order_items[revenue])

// 2. Total COGS
Total Cost = SUM(fact_order_items[cost])

// 3. Total Net Profit
Total Profit = [Total Revenue] - [Total Cost]

// 4. Gross Profit Margin % (With Safe Division)
Profit Margin % = DIVIDE([Total Profit], [Total Revenue], 0) * 100

// 5. Average Order Value (AOV)
Average Order Value = DIVIDE([Total Revenue], DISTINCTCOUNT(fact_orders[order_id]), 0)
```

Why DIVIDE() is Mandatory: Power BI me standard / division by zero par #ERROR deta hai jo dashboards ko break kar deta hai. DIVIDE(num, den, alternate_result) error-safe standard practice hai.

6.2 Advanced Time-Intelligence & Growth DAX Measures:

```
// 6. Year-to-Date (YTD) Revenue
Revenue YTD = TOTALYTD([Total Revenue], dim_date[Date])

// 7. Previous Month Revenue (Context Shift)
Previous Month Revenue =
CALCULATE(
    [Total Revenue],
    DATEADD(dim_date[Date], -1, MONTH)
)

// 8. Month-over-Month (MoM) Growth Percentage
MoM Revenue Growth % =
DIVIDE(
    [Total Revenue] - [Previous Month Revenue],
    [Previous Month Revenue],
    0
) * 100

// 9. Repeat Customers Count (Context Transition via CALCULATE inside FILTER)
Repeat Customers =
COUNTROWS(
    FILTER(
        VALUES(dim_customers[customer_id]),
        CALCULATE([Total Orders]) > 1
    )
)

// 10. Repeat Customer Share %
Repeat Customer % = DIVIDE([Repeat Customers], [Total Customers], 0) * 100
```

DAX Concept in Hinglish: CALCULATE() Power BI ka sabse powerful function hai jo **Filter Context** ko modify karta hai. VALUES(customer_id) unique customer list banata hai, aur FILTER() sirf unhe count karta hai jinka order count > 1 hai.

■ CHAPTER 7: Actionable Business Findings & Executive Recommendations

Is project se nikle 4 concrete business insights jo direct company ke bottom line profit ko optimize karte hain:

7.1 Finding 1: Discount Margin Cannibalization (The 12% Cliff)

- **Problem:** Data analysis ne prove kiya ki jab marketing team ne 15% se zyada discount offer kiya, toh realized profit margin **31.4% se drop hoke 14.2%** par aa gaya. Unit sales volume me sirf 8% ka increase hua jo margin drop ko compensate karne ke liye bilkul insufficient tha.
- **Recommendation:** Standard e-commerce promotional discount ko ****12% par cap**** kiya gaya. Discounts sirf slow-moving inventory clearance ke liye allow kiye gaye.
- **Dollar Impact:** Restored profit margin contributed an estimated **+\$210,000 annual net margin savings**.

7.2 Finding 2: Repeat Customer High-LTV Concentration (Pareto Principle)

- **Problem:** Repeat buyers total customer base ka sirf **41.8%** the, lekin unhone company ke total net profit ka **59.2% (\$1.35 Million)** generate kiya.
- **Behavioral Delta:** Repeat buyers ka Average Order Value (AOV) **\$582** tha compare to **\$312** for one-time buyers.
- **Recommendation:** Paid ad acquisition budget ka 25% re-allocate karke post-purchase automated email flows, VIP tier loyalty points, aur personalized product recommendation widgets me invest kiya gaya.

7.3 Finding 3: Delivery Lead Time vs Return Rate Threshold

- **Problem:** Orders jinka fulfillment transit time **5 din se zyada** tha unka customer return rate **22.4%** record hua, jabki <3 days me deliver hone wale orders ka return rate sirf **4.1%** tha.
- **Root Cause:** Slow transit customer buyer remorse aur competitive cancellation ko trigger karta hai.
- **Recommendation:** Regional 3PL fulfillment hubs establish kiye gaye jisse 85% shipments 48 hours ke andar deliver hone lagein.

7.4 Finding 4: Product Category Margin Dispersion

- **Electronics:** Highest GMV (\$4.1M, 49%) but moderate margin (22.8%) due to supplier hardware costs.
- **Office Supplies & Accessories:** Lower GMV (\$1.8M) but super-high margin (**41.2%**).
- **Recommendation:** Electronics ke sath high-margin accessories ka automated cross-sell bundle banaya gaya jisse basket margin 4.2% uplift hua.

■ CHAPTER 8: Complete Data Analyst Interview Playbook (10 Master Q&As;)

Q1: Explain your E-Commerce project in 60 seconds.

Ans: I developed an end-to-end data analytics pipeline analyzing 31,855 transactions (\$8.35M GMV). Using Python/Pandas for ETL, PostgreSQL for Star Schema data warehousing, and Power BI with 25+ DAX measures, I identified discount margin cannibalization (>15% discounts cut margin from 31% to 14%) and fulfillment delay return spikes (22.4%), providing actionable recommendations that saved \$210K annually.

Q2: Why did you choose Star Schema over 3NF or Snowflake?

Ans: 3NF is built for write-heavy OLTP databases to eliminate redundancy. For OLAP analytics, Star Schema de-normalizes dimensions, drastically reducing expensive multi-table joins, accelerating aggregations, and optimizing Power BI's in-memory columnar VertiPaq compression engine.

Q3: What is the difference between RANK(), DENSE_RANK(), and ROW_NUMBER()?

Ans: ROW_NUMBER() assigns strict sequential integers (1, 2, 3, 4). RANK() gives duplicate values identical ranks but skips subsequent ranks (1, 2, 2, 4). DENSE_RANK() assigns identical ranks without skipping (1, 2, 2, 3), which was ideal for our Category Top 3 Best-sellers analysis.

Q4: How did you compute Month-over-Month (MoM) Growth in SQL vs Power BI?

Ans: In SQL, I used a CTE with the LAG(monthly_revenue, 1) OVER (ORDER BY year_month) window function to fetch previous row values. In Power BI, I utilized CALCULATE([Total Revenue], DATEADD(dim_date[Date], -1, MONTH)) to shift the temporal filter context.

Q5: What was your strategy for handling missing values and data anomalies in Python?

Ans: In Python, I enforced pd.to_datetime(errors='coerce') to safely capture corrupt timestamp strings without pipeline crashes, filled non-critical categorical nulls with 'Unknown', and validated composite primary keys (order_id, product_id) to eliminate duplicate line items.

Q6: How does Row Context differ from Filter Context in DAX?

Ans: Row Context exists in calculated columns or iterator functions (SUMX, FILTER), evaluating expressions row-by-row. Filter Context is the combined set of active visual filters, slicers, and row/column coordinates. CALCULATE() is the only DAX function that transitions Row Context into Filter Context.

Q7: How did you calculate Customer Lifetime Value (LTV) and RFM Segmentation?

Ans: I built an RFM (Recency, Frequency, Monetary) query in SQL using NTILE(4) quartile distribution to categorize users into Champions, Loyalists, Potential At-Risk, and Churned cohorts, enabling targeted retention marketing.

Q8: If the CEO asks you 'How do we increase net profit by 10% next quarter?', what will you show?

Ans: I will open the Discount Elasticity & Customer Retention dashboard to demonstrate that capping discounts at 12% combined with re-engaging the top 41.8% repeat buyers will capture an immediate 6.4% margin uplift with zero additional ad spend.

Q9: What was the biggest technical roadblock you encountered and how did you solve it?

Ans: Handling mixed timestamp formats across multiple data sources and preventing circular filter dependencies in Power BI. I resolved this by enforcing strict ISO-8601 date parsing in Pandas and creating an explicit, disconnected Date dimension table with single-direction 1-to-Many relationships.

Q10: What makes a Data Analyst impactful in an e-commerce business?

Ans: Moving beyond descriptive reporting ("What happened?") to diagnostic and prescriptive analytics ("Why did it happen and what should we do next?"). Tying every metric directly to dollar impact, gross margin preservation, and customer retention.